



UNIVERSIDAD DE BUENOS AIRES
FACULTAD DE INGENIERÍA

86.41 - SISTEMAS DIGITALES

Trabajo Práctico N°3: CORDIC

BATALLAN, DAVID LEONARDO, *Padrón 97529*

28 DE JUNIO DE 2025

Índice

1. Introducción	2
2. Desarrollo	2
2.1. Especificaciones de diseño	2
2.2. Aclaraciones sobre el diseño	3
3. Implementación en VHDL	4
3.1. Cordic	4
3.2. Pre-Cordic	5
3.3. Arquitectura iterativa	6
3.4. Arquitectura desenrollada	8
3.5. Cordic Base	10
3.5.1. Componentes extras utilizados	11
4. Análisis de los Resultados	13
4.1. Arquitectura iterativa	15
4.2. Arquitectura desenrollada	15
5. Síntesis	16
5.1. Arquitectura iterativa	16
5.2. Arquitectura desenrollada	17
6. Conclusiones	18
7. Repositorio del Proyecto	19

1. Introducción

El presente trabajo tiene como objetivo describir e implementar en FPGA la arquitectura CORDIC. Dicha arquitectura será descrita en lenguaje VHDL, simulada mediante GTKWave y sintetizada sobre el dispositivo xc7a15tftg256-1.

El algoritmo CORDIC es ampliamente utilizado para la rotación de vectores, el cálculo de funciones trigonométricas, la transformación de coordenadas, entre otras aplicaciones. En el caso particular de este trabajo, se utilizará exclusivamente para la rotación de vectores.

2. Desarrollo

El algoritmo CORDIC en este trabajo será utilizado para la rotación de vectores en el plano XY. Este algoritmo cuenta con dos modos de operación principales: rotación y vectorización.

En el modo rotación, el vector se gira un ángulo especificado previamente. El acumulador angular se inicializa con el valor de dicho ángulo, y la decisión sobre la dirección de rotación en cada iteración se realiza de manera que el ángulo residual en el acumulador disminuya, utilizando el signo del mismo para determinar el sentido de rotación.

Este modo puede resumirse mediante las siguientes ecuaciones:

$$\begin{cases} x_{i+1} = x_i - y_i \cdot d_i \cdot 2^{-i} \\ y_{i+1} = y_i + x_i \cdot d_i \cdot 2^{-i} \\ z_{i+1} = z_i - d_i \cdot \arctan(2^{-i}) \end{cases} \quad (1)$$

Donde:

$$d_i = \begin{cases} -1 & \text{si } z_i < 0, \\ +1 & \text{en otro caso.} \end{cases} \quad (2)$$

Y finalmente, se obtiene:

$$\begin{cases} x_n = A_n (x_0 \cdot \cos(z_0) - y_0 \cdot \sin(z_0)) \\ y_n = A_n (y_0 \cdot \cos(z_0) + x_0 \cdot \sin(z_0)) \\ z_n = 0 \end{cases} \quad (3)$$

En cambio, en el modo vectorización, el algoritmo rota un vector hacia el eje de coordenadas x , almacenando los ángulos requeridos para lograr dicha rotación. Este modo busca minimizar la componente y del vector residual, y la dirección de rotación en cada iteración se determina según el signo de la componente y residual.

Este modo puede resumirse mediante las siguientes ecuaciones:

$$\begin{cases} x_{i+1} = x_i - y_i \cdot d_i \cdot 2^{-i} \\ y_{i+1} = y_i + x_i \cdot d_i \cdot 2^{-i} \\ z_{i+1} = z_i - d_i \cdot \arctan(2^{-i}) \end{cases} \quad (4)$$

Donde:

$$d_i = \begin{cases} +1 & \text{si } y_i < 0, \\ -1 & \text{en otro caso.} \end{cases} \quad (5)$$

Y finalmente se tiene:

$$\begin{cases} x_n = A_n \cdot \sqrt{x_0^2 + y_0^2} \\ y_n = 0 \\ z_n = z_0 + \arctan\left(\frac{y_0}{x_0}\right) \end{cases} \quad (6)$$

El A_n mencionado en las ecuaciones 3 y 6 es la ganancia del CORDIC, la cual vale aproximadamente 1.647 cuando la cantidad de iteraciones es lo suficientemente grande.

2.1. Especificaciones de diseño

Se implementaron, en lenguaje descriptor de hardware VHDL, dos arquitecturas CORDIC trabajando en ambos modos: rotación y vectorización. En la siguiente figura se puede observar un diagrama en bloques aproximado del diseño.

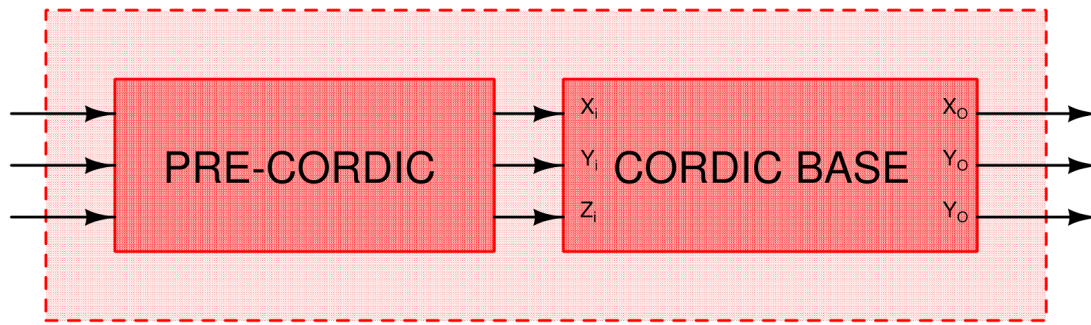


Figura 1: Diagrama en bloques

El bloque pre-CORDIC es una lógica combinacional que se encarga de, en modo rotación, negar las coordenadas x y y originales para poder rotar ángulos mayores a 90° . Para el modo vectorización, este bloque no tiene funcionalidad alguna.

La primera arquitectura es iterativa y la segunda, desenrollada. A esta última se le agregó pipelining en las etapas intermedias.

En las siguientes figuras se muestran los diagramas de estas arquitecturas.

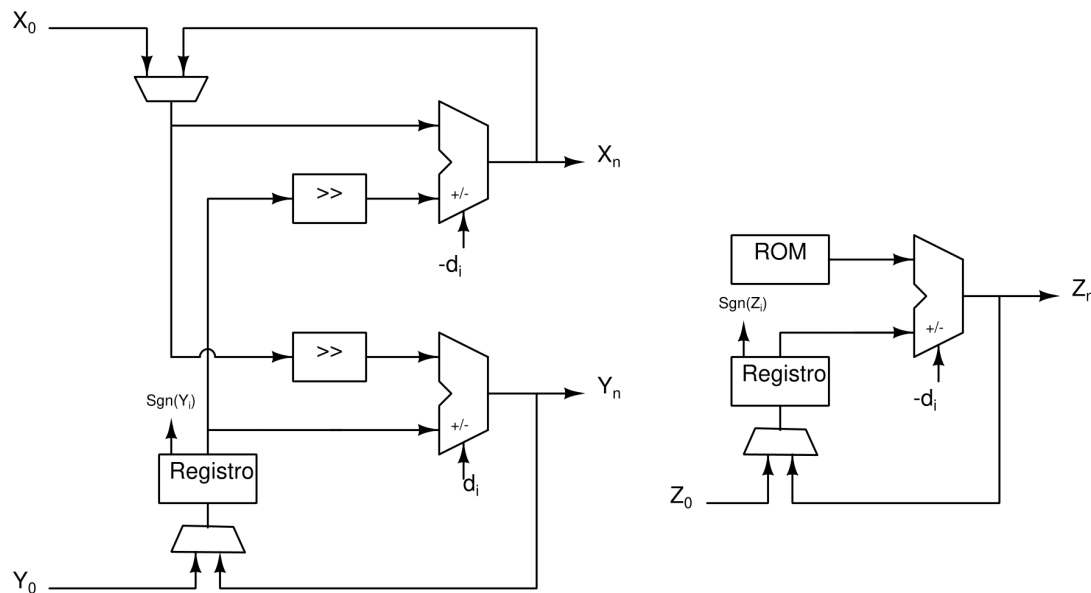


Figura 2: Arquitectura iterativa

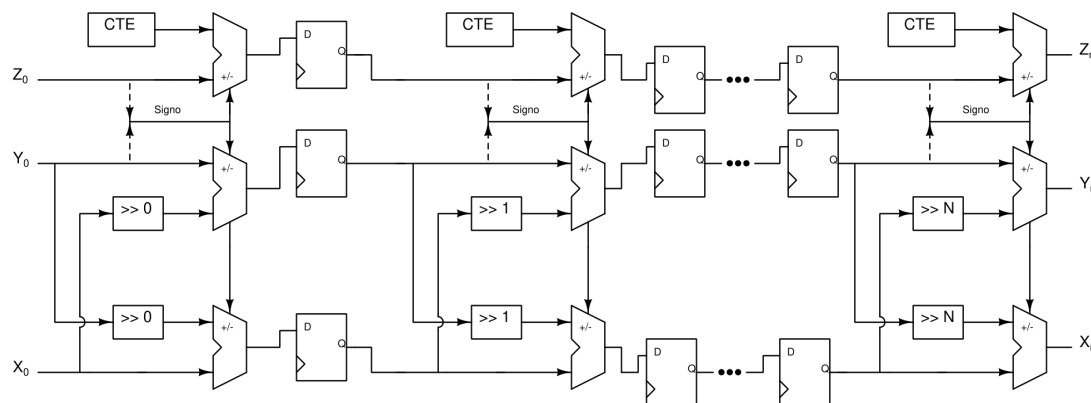


Figura 3: Arquitectura desenrollada con pipelining

2.2. Aclaraciones sobre el diseño

Se eligió realizar 16 iteraciones del algoritmo, por lo que se utilizarán 16 bits para los datos. Además, se seleccionó un fondo de escala de 2^{13} , por lo que tanto los ángulos como las coordenadas de los vectores estarán escaladas por

este valor. Es decir,

$$\begin{cases} \text{Coordenadas:} & x = x \cdot 2^{13} \\ \text{Ángulos:} & \text{Ang} = \frac{\text{Ang}}{\arctan(1)} \cdot 2^{13} \end{cases} \quad (7)$$

También se decidió no utilizar un bloque post-CORDIC. Este bloque se encargaría de compensar la ganancia del CORDIC en las coordenadas finales, pero al no requerirse esta funcionalidad, se optó por no implementarlo.

3. Implementación en VHDL

A continuación se muestra el código utilizado para implementar ambas arquitecturas en VHDL.

3.1. Cordic

CORDIC.VHD

```

1  -- TP3
2  -- Materia: Sistemas digitales
3  -- Alumno: Batallan David Leonardo
4  -- Padron: 97529
5
6  library ieee;
7  use ieee.std_logic_1164.all;
8  use ieee.numeric_std.all;
9  use ieee.math_real.all;
10
11 entity cordic is
12     generic(
13         N : natural := 16
14     );
15     port(
16         clk          : in std_logic;
17         rst          : in std_logic;
18         req          : in std_logic;
19         rot0_vec1    : in std_logic;
20         x_i, y_i, z_i : in std_logic_vector(N-1 downto 0);
21         ack          : out std_logic;
22         x_o, y_o, z_o : out std_logic_vector(N-1 downto 0)
23     );
24 end cordic;
25
26
27 architecture behavioral_iter of cordic is
28
29     -- Entradas y salidas de Pre - Cordic
30     signal x_i_precordic, y_i_precordic, z_i_precordic : std_logic_vector(N-1 downto 0);
31     signal x_o_precordic, y_o_precordic, z_o_precordic : std_logic_vector(N-1 downto 0);
32
33     -- Entradas y salidas de Cordic_iterativo
34     signal x_i_cordic, y_i_cordic, z_i_cordic : std_logic_vector(N-1 downto 0);
35     signal ack_aux : std_logic;
36
37
38 begin
39
40     x_i_precordic <= x_i;
41     y_i_precordic <= y_i;
42     z_i_precordic <= z_i;
43     ack <= ack_aux;
44
45     PRE_CORDIC: entity work.pre_cordic(behavioral)
46     generic map(N => N)
47     port map(
48         x_i => x_i_precordic,
49         y_i => y_i_precordic,
50         z_i => z_i_precordic,
51         rot0_vec1 => rot0_vec1,
52         x_o => x_o_precordic,
53         y_o => y_o_precordic,
54         z_o => z_o_precordic
55     );
56
57     CORDIC_I: entity work.cordic_iter(behavioral)

```

```

58     generic map(N => N)
59     port map(
60         clk      => clk,
61         rst      => rst,
62         req      => req,
63         ack      => ack_aux,
64         rot0_vec1 => rot0_vec1,
65         x_0      => x_o_precordic,
66         y_0      => y_o_precordic,
67         z_0      => z_o_precordic,
68         x_nm1    => x_o,
69         y_nm1    => y_o,
70         z_nm1    => z_o
71     );
72
73 end behavioral_iter;
74
75
76 architecture behavioral_unrolled of cordic is
77
78     -- Entradas y salidas de Pre - Cordic
79     signal x_i_precordic, y_i_precordic, z_i_precordic : std_logic_vector(N-1 downto 0);
80     signal x_o_precordic, y_o_precordic, z_o_precordic : std_logic_vector(N-1 downto 0);
81
82     -- Entradas y salidas de Cordic_iterativo
83     signal x_i_cordic, y_i_cordic, z_i_cordic : std_logic_vector(N-1 downto 0);
84     signal ack_aux : std_logic;
85
86
87
88 begin
89
90     x_i_precordic <= x_i;
91     y_i_precordic <= y_i;
92     z_i_precordic <= z_i;
93     ack <= ack_aux;
94
95
96     PRE_CORDIC: entity work.pre_cordic(behavioral)
97     generic map(N => N)
98     port map(
99         x_i      => x_i_precordic,
100        y_i      => y_i_precordic,
101        z_i      => z_i_precordic,
102        rot0_vec1 => rot0_vec1,
103        x_o      => x_o_precordic,
104        y_o      => y_o_precordic,
105        z_o      => z_o_precordic
106    );
107
108     CORDIC_UP: entity work.cordic_unrolled(behavioral)
109     generic map(
110         N => N
111     )
112     port map(
113         clk => clk,
114         rst => rst,
115         req => req,
116         ack => ack_aux,
117         rot0_vec1 => rot0_vec1,
118         x_0 => x_o_precordic,
119         y_0 => y_o_precordic,
120         z_0 => z_o_precordic,
121         x_nm1 => x_o,
122         y_nm1 => y_o,
123         z_nm1 => z_o
124     );
125
126 end behavioral_unrolled;

```

Listing 1: Código VHDL de cordic

3.2. Pre-Cordic

PRE_CORDIC.VHD

```

1  -- TP3
2  -- Materia: Sistemas digitales
3  -- Alumno: Batallan David Leonardo
4  -- Padron: 97529
5
6  library ieee;
7  use ieee.std_logic_1164.all;
8  use ieee.numeric_std.all;
9
10 entity pre_cordic is
11     generic(
12         N : natural := 16 -- cantidad de iteraciones que va a hacer el algoritmo
13     );
14     port(
15
16         x_i      : in std_logic_vector(N-1 downto 0);
17         y_i      : in std_logic_vector(N-1 downto 0);
18         z_i      : in std_logic_vector(N-1 downto 0);
19         rot0_vec1 : in std_logic;
20         x_o      : out std_logic_vector(N-1 downto 0);
21         y_o      : out std_logic_vector(N-1 downto 0);
22         z_o      : out std_logic_vector(N-1 downto 0)
23     );
24
25 end pre_cordic;
26
27 architecture behavioral of pre_cordic is
28
29     -- 2**(N-1) --> 180
30     -- 2**(N-2) --> 90
31     -- 2**(N-3) --> 45
32     constant ANG_180 : signed := to_signed(2**(N-1), N);
33     constant ANG_90  : signed := to_signed(2**(N-2), N);
34     constant ANG_45  : signed := to_signed(2**(N-3), N);
35     signal x_o_rot0, y_o_rot0, z_o_rot0 : std_logic_vector(N-1 downto 0);
36     signal x_o_vec1, y_o_vec1, z_o_vec1 : std_logic_vector(N-1 downto 0);
37
38 begin
39
40     x_o_vec1 <= x_i;
41     y_o_vec1 <= y_i;
42     z_o_vec1 <= z_i;
43
44
45     x_o_rot0 <= std_logic_vector(signed(not(x_i)) + 1) when std_logic_vector(abs(signed(z_i))) >
46     ⇨ std_logic_vector(abs(ANG_90)) else
47         x_i;
48
49     y_o_rot0 <= std_logic_vector(signed(not(y_i)) + 1) when std_logic_vector(abs(signed(z_i))) >
50     ⇨ std_logic_vector(abs(ANG_90)) else
51         y_i;
52
53     z_o_rot0 <= std_logic_vector( signed(z_i) - ANG_180 ) when signed(z_i) > ANG_90 else
54         std_logic_vector( signed(z_i) + ANG_180 ) when signed(z_i) < -ANG_90 else
55         z_i;
56
57     x_o <= x_o_rot0 when rot0_vec1 = '0' else
58         x_o_vec1;
59
60     y_o <= y_o_rot0 when rot0_vec1 = '0' else
61         y_o_vec1;
62
63     z_o <= z_o_rot0 when rot0_vec1 = '0' else
64         z_o_vec1;
65
66 end behavioral;

```

3.3. Arquitectura iterativa

CORDIC_ITER.VHD

```

1  -- TP3
2  -- Materia: Sistemas digitales
3  -- Alumno: Batallan David Leonardo
4  -- Padron: 97529
5
6  library ieee;
7  use ieee.std_logic_1164.all;
8  use ieee.numeric_std.all;
9  use ieee.math_real.all;
10
11 entity cordic_iter is
12   generic(
13     N : natural := 16 -- cantidad de iteraciones que va a hacer el algoritmo
14   );
15   port(
16     clk: in std_logic;
17     rst: in std_logic;
18     req: in std_logic;
19     ack: out std_logic;
20     rot0_vec1: in std_logic;
21     x_0 : in std_logic_vector(N-1 downto 0);
22     y_0 : in std_logic_vector(N-1 downto 0);
23     z_0 : in std_logic_vector(N-1 downto 0);
24     x_nm1 : out std_logic_vector(N-1 downto 0);
25     y_nm1 : out std_logic_vector(N-1 downto 0);
26     z_nm1 : out std_logic_vector(N-1 downto 0)
27   );
28 end cordic_iter;
29
30 architecture behavioral of cordic_iter is
31
32   constant CLOG2N : natural := natural(ceil(log2(real(N))));
33
34   signal count : unsigned(CLOG2N-1 downto 0);
35
36   signal x_i,y_i,z_i : std_logic_vector(N-1 downto 0);
37   signal x_ip1,y_ip1,z_ip1 : std_logic_vector(N-1 downto 0);
38   signal atan_2mi : std_logic_vector(N-3 downto 0);
39   signal ack_aux : std_logic;
40
41 begin
42
43   ROM:entity work.ATAN_ROM(behavioral)
44     generic map(
45       ADD_W => CLOG2N,
46       DATA_W => N-2
47     )
48     port map(
49       addr_i => std_logic_vector(count),
50       data_o => atan_2mi
51     );
52
53
54   CORDIC_BASE: entity work.cordic_base(behavioral)
55     generic map(
56       N => N,
57       CLOG2N => CLOG2N
58     )
59     port map(
60       num_iter => count,
61       rot0_vec1 => rot0_vec1,
62       x_i => x_i,
63       y_i => y_i,
64       z_i => z_i,
65       atan_2mi => atan_2mi,
66       x_ip1 => x_ip1,
67       y_ip1 => y_ip1,
68       z_ip1 => z_ip1
69     );
70
71   process(clk,rst)
72   begin
73     if rst = '1' then
74       count <= (others => '0');
75     elsif clk'event and clk = '1' then
76       if req = '1' then

```



```

77         count <= (others => '0');
78     elsif count /= N-1 then
79         count <= count + 1;
80     end if;
81 end if;
82 end process;
83
84 ack_aux <= '1' when count = N-1 else
85     '0';
86
87 ack <= ack_aux;
88
89 process(clk,rst)
90 begin
91     if rst = '1' then
92         x_i <= (others => '0');
93         y_i <= (others => '0');
94         z_i <= (others => '0');
95     elsif clk'event and clk = '1' then
96         if req = '1' then
97             x_i <= x_0;
98             y_i <= y_0;
99             z_i <= z_0;
100         else
101             x_i <= x_ip1;
102             y_i <= y_ip1;
103             z_i <= z_ip1;
104         end if;
105     end if;
106 end process;
107
108 x_nm1 <= x_ip1 when ack_aux = '1' else (others => '0');
109 y_nm1 <= y_ip1 when ack_aux = '1' else (others => '0');
110 z_nm1 <= z_ip1 when ack_aux = '1' else (others => '0');
111
112 end behavioral;

```

3.4. Arquitectura desenrollada

CORDIC_UNROLLED.VHD

```

1  -- TP3
2  -- Materia: Sistemas Digitales
3  -- Alumno: David Leonardo Batallan
4  -- Padrón: 97529
5
6  library ieee;
7  use ieee.std_logic_1164.all;
8  use ieee.numeric_std.all;
9  use ieee.math_real.all;
10
11 entity cordic_unrolled is
12     generic(
13         N : natural := 16
14     );
15     port(
16         clk: in std_logic;
17         rst: in std_logic;
18         req: in std_logic;
19         ack: out std_logic;
20         rot0_vec1: in std_logic; -- Señal para seleccionar modo rotación o vectorial
21         x_0 : in std_logic_vector(N-1 downto 0);
22         y_0 : in std_logic_vector(N-1 downto 0);
23         z_0 : in std_logic_vector(N-1 downto 0);
24         x_nm1 : out std_logic_vector(N-1 downto 0);
25         y_nm1 : out std_logic_vector(N-1 downto 0);
26         z_nm1 : out std_logic_vector(N-1 downto 0)
27     );
28 end cordic_unrolled;
29
30 architecture behavioral of cordic_unrolled is
31
32     type matrix_type is array (natural range <>) of std_logic_vector(N-1 downto 0);

```

```

33
34     constant CLOG2N : natural := natural(ceil(log2(real(N)))); -- Cantidad de bits necesarios para contar hasta N
35
36     signal count : unsigned(CLOG2N-1 downto 0) := (others => '0');
37     signal x_vect, y_vect, z_vect : matrix_type(N downto 0);
38     signal x_reg, y_reg, z_reg : matrix_type(N-1 downto 0);
39     signal ack_aux : std_logic;
40     signal atan_2mi : std_logic_vector(N-3 downto 0);
41
42
43 begin
44
45     -- Se intentó calcular arctangentes en tiempo de ejecución, pero Vivado no permite usar funciones de punto flotante
46     -- ↪ no constantes.
47     -- Por lo tanto, se utiliza una ROM para almacenar los valores precomputados de atan(2-i)
48
49     ROM: entity work.ATAN_ROM(behavioral)
50         generic map(
51             ADD_W => CLOG2N,
52             DATA_W => N-2
53         )
54         port map(
55             addr_i => std_logic_vector(count),
56             data_o => atan_2mi
57         );
58
59     -- Generación de N etapas de la arquitectura CORDIC desenrollada
60     CORDIC_UNR: for i in 0 to N-1 generate
61
62         CORDIC_BASE: entity work.cordic_base(behavioral)
63             generic map(
64                 N => N,
65                 CLOG2N => CLOG2N
66             )
67             port map(
68                 num_iter => count,
69                 rot0_vec1 => rot0_vec1,
70                 x_i => x_vect(i),
71                 y_i => y_vect(i),
72                 z_i => z_vect(i),
73                 atan_2mi => atan_2mi,
74                 x_ip1 => x_reg(i),
75                 y_ip1 => y_reg(i),
76                 z_ip1 => z_reg(i)
77             );
78
79     end generate CORDIC_UNR;
80
81     -- Registro intermedio para señal x
82     REGISTER_X: for i in 0 to N-1 generate
83         REG_X: entity work.reg(behavioral)
84             generic map(
85                 N => N
86             )
87             port map(
88                 d => x_reg(i),
89                 rst => rst,
90                 clk => clk,
91                 q => x_vect(i+1)
92             );
93     end generate REGISTER_X;
94
95     -- Registro intermedio para señal y
96     REGISTER_Y: for i in 0 to N-1 generate
97         REG_Y: entity work.reg(behavioral)
98             generic map(
99                 N => N
100             )
101             port map(
102                 d => y_reg(i),
103                 rst => rst,
104                 clk => clk,
105                 q => y_vect(i+1)
106             );
107     end generate REGISTER_Y;

```

```

108  -- Registro intermedio para señal z
109  REGISTER_Z: for i in 0 to N-1 generate
110      REG_Z: entity work.reg(behavioral)
111          generic map(
112              N => N
113          )
114          port map(
115              d => z_reg(i),
116              rst => rst,
117              clk => clk,
118              q => z_vect(i+1)
119          );
120  end generate REGISTER_Z;
121
122  -- Asignación de entradas iniciales
123  x_vect(0) <= x_0;
124  y_vect(0) <= y_0;
125  z_vect(0) <= z_0;
126
127  -- Asignación de salidas finales luego de N iteraciones
128  x_nm1 <= x_reg(N-1);
129  y_nm1 <= y_reg(N-1);
130  z_nm1 <= z_reg(N-1);
131
132  -- Lógica de control para contador de etapas e inicio del proceso
133  process(clk,rst)
134  begin
135      if rst = '1' then
136          count <= (others => '0');
137      elsif clk'event and clk = '1' then
138          if req = '1' then
139              count <= (others => '0');
140          elsif count /= N-1 then
141              count <= count + 1;
142          end if;
143      end if;
144  end process;
145
146  -- Señal de finalización de la operación
147  ack_aux <= '1' when count = N-1 else '0';
148  ack <= ack_aux;
149
150  end behavioral;

```

3.5. Cordic Base

CORDIC_BASE.VHD

```

1  -- TP3
2  -- Materia: Sistemas digitales
3  -- Alumno: Batallan David Leonardo
4  -- Padron: 97529
5
6  library ieee;
7  use ieee.std_logic_1164.all;
8  use ieee.numeric_std.all;
9
10 entity cordic_base is
11     generic(
12         N : natural := 16; -- cantidad de iteraciones que va a hacer el algoritmo
13         CLOG2N : natural := 4 -- tamaño del punto fijo
14     );
15     port(
16         num_iter : in unsigned(CLOG2N-1 downto 0);
17         rot0_vec1 : in std_logic;
18         x_i : in std_logic_vector(N-1 downto 0);
19         y_i : in std_logic_vector(N-1 downto 0);
20         z_i : in std_logic_vector(N-1 downto 0);
21         atan_2mi : in std_logic_vector(N-3 downto 0);
22         x_ip1 : out std_logic_vector(N-1 downto 0);
23         y_ip1 : out std_logic_vector(N-1 downto 0);
24         z_ip1 : out std_logic_vector(N-1 downto 0)
25     );

```

```

26 end cordic_base;
27
28 architecture behavioral of cordic_base is
29
30     signal x_aux,y_aux,z_aux : std_logic_vector(N-1 downto 0);
31     signal x_shifted,y_shifted : std_logic_vector(N-1 downto 0);
32     signal x_o,y_o,z_o : std_logic_vector(N-1 downto 0);
33     signal di: std_logic;
34     signal not_di: std_logic;
35     signal sx_atan_2mi: std_logic_vector(N-1 downto 0);
36
37 begin
38
39     x_aux <= x_i;
40     y_aux <= y_i;
41     z_aux <= z_i;
42
43     x_shifted <= std_logic_vector(shift_right(signed(x_aux), to_integer(num_iter)));
44     y_shifted <= std_logic_vector(shift_right(signed(y_aux), to_integer(num_iter)));
45
46     di <= z_i(N-1) when rot0_vec1 = '0' else
47         not(y_i(N-1));
48
49     not_di <= not(di);
50
51     sumador_x: entity work.add_sub(behavioral)
52     generic map(N => N)
53     port map(
54         x => x_aux,
55         y => y_shifted,
56         z => x_o,
57         add0_sub1 => not_di
58     );
59
60     sumador_y: entity work.add_sub(behavioral)
61     generic map(N => N)
62     port map(
63         x => y_aux,
64         y => x_shifted,
65         z => y_o,
66         add0_sub1 => di
67     );
68
69     sx_atan_2mi <= "00" & atan_2mi;
70
71     sumador_z: entity work.add_sub(behavioral)
72     generic map(N => N)
73     port map(
74         x => z_aux,
75         y => sx_atan_2mi,
76         z => z_o,
77         add0_sub1 => not_di
78     );
79
80     x_ip1 <= x_o;
81     y_ip1 <= y_o;
82     z_ip1 <= z_o;
83
84 end behavioral;

```

3.5.1. Componentes extras utilizados

ATAN__ROM.VHD

```

1  -- TP3
2  -- Materia: Sistemas digitales
3  -- Alumno: Batallan David Leonardo
4  -- Padron: 97529
5
6  library IEEE;
7  use IEEE.std_logic_1164.all;
8  use IEEE.numeric_std.all;
9  use IEEE.math_real.all;

```

```

10
11 entity atan_rom is
12     generic(
13         ADD_W  : natural := 10;
14         DATA_W : natural := 9
15     );
16     port(
17         addr_i : in std_logic_vector(ADD_W-1 downto 0);
18         data_o : out std_logic_vector(DATA_W-1 downto 0)
19     );
20 end entity atan_rom;
21
22
23 architecture behavioral of atan_rom is
24
25     type rom_type is array (natural range <>) of std_logic_vector(DATA_W-1 downto 0);
26
27     function atan (constant i : natural) return std_logic_vector is
28
29     begin
30         return std_logic_vector(to_unsigned(integer(round( (arctan(real(2)**real(-1*i)) / arctan(real(1))) *
31             ↪ real(2**(DATA_W-1)) )), DATA_W));
32     end;
33
34     signal rom : rom_type(0 to 2**ADD_W-1);
35
36 begin
37     Load_ROM : for i in 0 to DATA_W-1 generate
38         rom(i) <= atan(i);
39     end generate Load_ROM;
40
41     data_o <= (others => '0') when unsigned(addr_i) > to_unsigned(DATA_W-1, ADD_W) else
42         rom(to_integer(unsigned(addr_i)));
43
44 end behavioral;

```

ADD_SUB.VHD

```

1  -- TP3
2  -- Materia: Sistemas digitales
3  -- Alumno: Batallan David Leonardo
4  -- Padron: 97529
5
6  library IEEE;
7  use IEEE.std_logic_1164.all;
8  use IEEE.numeric_std.all;
9
10 entity add_sub is
11     generic(N : natural := 5);
12     port(
13         x      : in std_logic_vector(N-1 downto 0);
14         y      : in std_logic_vector(N-1 downto 0);
15         add0_sub1 : in std_logic;
16         z : out std_logic_vector(N-1 downto 0)
17     );
18 end add_sub;
19
20 architecture behavioral of add_sub is
21
22 begin
23     z <= std_logic_vector(signed(x) + signed(y)) when add0_sub1 = '0' else
24         std_logic_vector(signed(x) - signed(y));
25
26 end behavioral;

```

REGISTER.VHD

```

1  -- TP3
2  -- Materia: Sistemas digitales
3  -- Alumno: Batallan David Leonardo

```

```

4  -- Padron: 97529
5
6  library ieee;
7  use ieee.std_logic_1164.all;
8
9  entity reg is
10     generic(
11         N : natural := 16
12     );
13     port(
14         d  : in std_logic_vector(N-1 downto 0);
15         rst : in std_logic;
16         clk : in std_logic;
17         q  : out std_logic_vector(N-1 downto 0)
18     );
19
20 end reg;
21
22 architecture behavioral of reg is
23
24 begin
25
26     process(clk,rst)
27     begin
28         if rst = '1' then
29             q <= (others => '0');
30
31         elsif clk = '1' and clk'event then
32             q <= d;
33
34         end if;
35     end process;
36
37 end behavioral;

```

4. Análisis de los Resultados

Para simular estas arquitecturas se utilizó el software Excel para comparar los resultados arrojados en la simulación, con las ecuaciones 3 y 6. Tanto para la arquitectura iterativa como desenrollada se decidió utilizar los siguientes parámetros, teniendo en cuenta que las coordenadas en realidad se consideran a fondo de escala, es decir, en este caso la coordenada 0,5 sería 4096.

$$\begin{cases} \text{Coordenadas 1: } (x_0, y_0) = \{0, 5; 0\} & \text{Ángulo a rotar: } \beta = 45^\circ \\ \text{Coordenadas 2: } (x_0, y_0) = \{0, 5; 0, 5\} & \text{Ángulos a rotar: } \beta_1 = 150^\circ, \quad \beta_2 = 200^\circ, \quad \beta_3 = -128^\circ \end{cases} \quad (8)$$

Además, para ambas arquitecturas se utilizó el siguiente banco de pruebas

CORDIC_TB.VHD

```

1  -- TP3
2  -- Materia: Sistemas digitales
3  -- Alumno: Batallan David Leonardo
4  -- Padron: 97529
5
6  library ieee;
7  use ieee.std_logic_1164.all;
8  use ieee.numeric_std.all;
9  use ieee.math_real.all;
10 use std.textio.all;
11
12 entity cordic_testbench is
13 end entity cordic_testbench;
14
15 architecture cordic_testbench_arq of cordic_testbench is
16
17     constant N : natural := 16;
18     constant FILE_PATH : string := "..\Datos.txt";
19
20     signal clk : std_logic := '0';

```

```

21  signal rst : std_logic := '1';
22  signal req : std_logic := '0';
23  signal ack : std_logic;
24  signal rot0_vec1 : std_logic := '0';
25  signal x_in,y_in,z_in : std_logic_vector(N-1 downto 0);
26  signal x_out,y_out,z_out : std_logic_vector(N-1 downto 0);
27
28  file datos: text open read_mode is FILE_PATH;
29
30
31  begin
32
33  clk <= not clk after 10 us;
34  rst <= '0' after 2 us;
35
36  Test_Sequence: process
37
38      variable l : line;
39      variable ch : character := ' ';
40      variable aux : integer;
41      variable z_file: integer;
42      variable ANG_RAD: real;
43      variable ANG_Z : integer;
44
45  begin
46
47      while not(endfile(datos)) loop
48          wait until rising_edge(clk);
49          -- Se lee una línea del archivo de valores de prueba
50          readline(datos, l);
51          -- Se extrae un entero de la línea
52          read(l, aux);
53          -- Se carga el valor de la coordenada X (en fondo de escala)
54          x_in <= std_logic_vector(to_unsigned(aux, N));
55          -- Se lee un caracter (el espacio)
56          read(l, ch);
57          -- Se lee otro entero de la línea
58          read(l, aux);
59          -- Se carga el valor de la coordenada Y (en fondo de escala)
60          y_in <= std_logic_vector(to_signed(aux, N));
61          -- Se lee otro caracter (el espacio)
62          read(l, ch);
63          -- Se lee otro entero
64          read(l, aux);
65          -- Se carga el valor del angulo a rotar (en grados)
66          z_file := aux;
67
68          -- Opero con el angulo a rotar.
69          ANG_RAD := (real(z_file)*MATH_PI)/real(180); -- Lo paso a radianes
70          ANG_Z := integer( round( (ANG_RAD/arctan(real(1))) * real(2**(N-3)) ) ); -- Lo escalo
71
72          -- Se carga el valor correspondiente del angulo a rotar
73          z_in <= std_logic_vector(to_signed(ANG_Z,N));
74
75          req <= '1';
76          wait until rising_edge(clk);
77          req <= '0';
78          wait until ack = '1';
79          wait until rising_edge(clk);
80      end loop;
81
82      file_close(datos); -- Se cierra el archivo
83
84      -- Se aborta la simulacion (fin del archivo)
85      assert false report
86          "Fin de la simulacion" severity failure;
87
88  end process Test_Sequence;
89
90  DUT: entity work.cordic(behavioral_unrolled)
91  generic map(
92      N => N
93  )
94  port map(
95      clk => clk,
96      rst => rst,

```

```

97     req => req,
98     rot0_vec1 => rot0_vec1,
99     x_i => x_in,
100    y_i => y_in,
101    z_i => z_in,
102    ack => ack,
103    x_o => x_out,
104    y_o => y_out,
105    z_o => z_out
106
107 );
108
109 end architecture cordic_testbench_arq;

```

4.1. Arquitectura iterativa

Los resultados de Excel para el primer caso, donde los parámetros son $(x_0, y_0) = \{0, 5; 0\}$ y el ángulo a rotar es $\beta = 45^\circ$, pueden verse en la siguiente figura.

CORDIC - ALGORITMO PARA ROTACION DE VECTORES

Xin	0,5	4096
Yin	0	0
Zin [°]	45	16384
Zin [rad]	0,785398163	

Modo ROT

Xout	4769,352649
Yout	4769,352649
Zout [°]	0

Modo Vec

Xout	6744,6832
Yout	0
Zout [°]	8192

G Cordic	1,6487
----------	--------

Unidad	1	8192
--------	---	------

Figura 4: Resultados Excel

El resultado (para modo rotación) de la simulación, junto con la simulación completa en el software GTKwave, pueden verse en las figuras 5 y 6 respectivamente. El resto de los resultados se pueden comprobar ejecutando la simulación con el banco de pruebas indicado anteriormente.

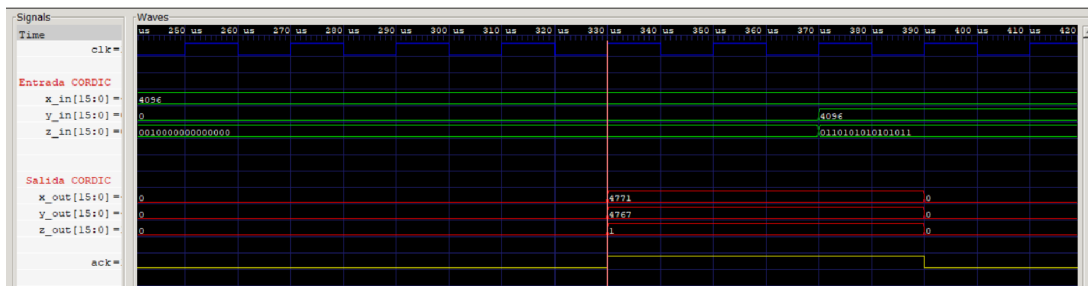


Figura 5: Resultados simulación

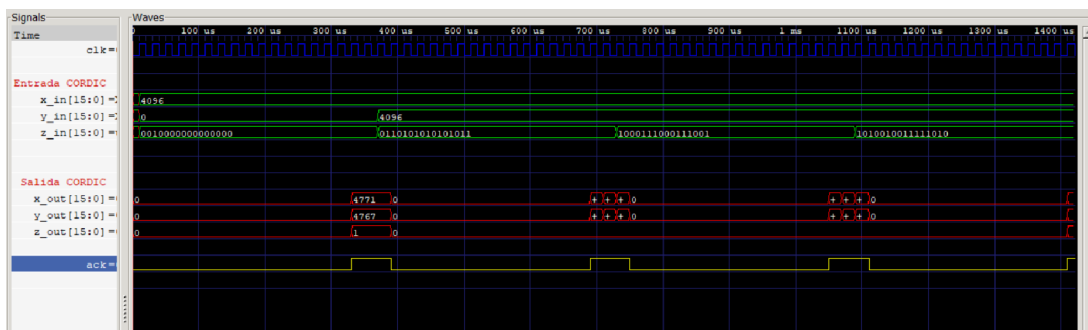


Figura 6: Simulación completa

4.2. Arquitectura desenrollada

Los resultados obtenidos en Excel para el caso en que los parámetros son $(x_0, y_0) = \{0, 5; 0, 5\}$ y el ángulo a rotar es $\beta = -128^\circ$, pueden verse en la siguiente figura.

CORDIC - ALGORITMO PARA ROTACION DE VECTORES

Xin	0,5	4096
Yin	0,5	4096
Zin [°]	-128	-46603
Zin [rad]	-2,234021443	

Modo ROT

Xout	1162,475752
Yout	-9467,805235
Zout [°]	0

Modo Vec

Xout	9538,705298
Yout	0
Zout [°]	-15109,68889

G Cordic	1,6467
----------	--------

Unidad	1	8192
--------	---	------

Figura 7: Resultados Excel

El resultado correspondiente al modo rotación, así como la simulación completa en el software *GTKWave*, pueden observarse en las Figuras 8 y ??, respectivamente. El resto de los resultados puede comprobarse ejecutando la simulación con el banco de pruebas mencionado anteriormente.

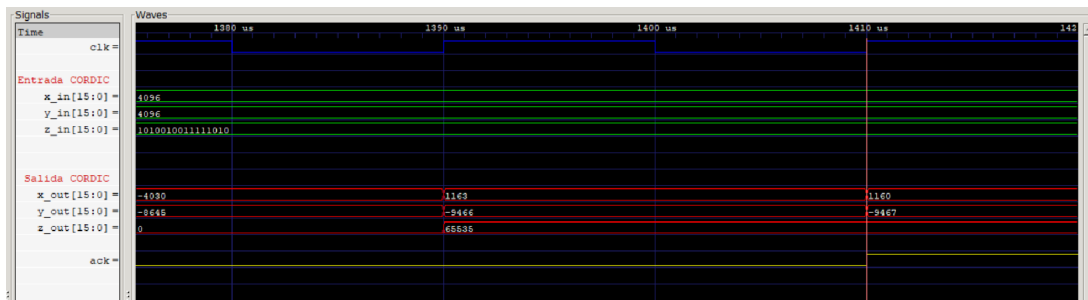


Figura 8: Resultados simulación

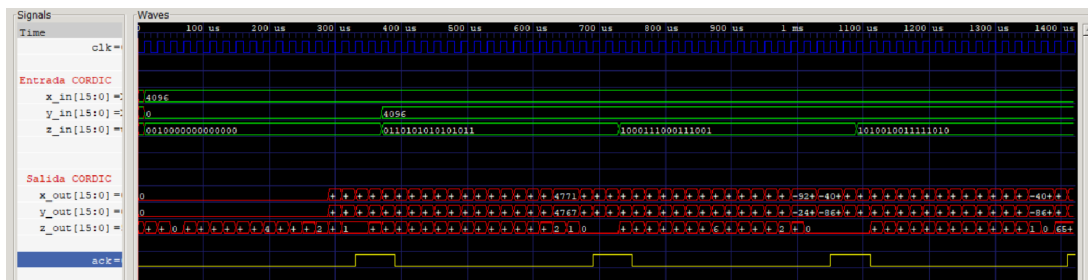


Figura 9: Simulación completa

5. Síntesis

Para esta sección, se realizó una síntesis sobre el dispositivo FPGA xc7a15tftg256-1 mediante el software Vivado. En esta herramienta, se muestra como resulta el circuito descrito mediante VHDL implementado en el dispositivo.

5.1. Arquitectura iterativa

El esquemático RTL se muestra en las Figuras 10 y 11, siendo esta última un acercamiento que permite visualizar con mayor detalle la entidad *CORDIC_I*. En cuanto al circuito completo de implementación, su tamaño excede lo conveniente para ser incluido en el cuerpo principal del informe, por lo que se adjunta como anexo al presente trabajo.

Otro de los resultados de interés es el resumen de recursos utilizados en el diseño, donde se detalla la cantidad de *Flip-Flops*, *LUTs* y puertos de entrada/salida necesarios para su implementación en el dispositivo. Dicho resumen, junto con el resumen de tiempos, se presentan en las Figuras 12 y 13, respectivamente.

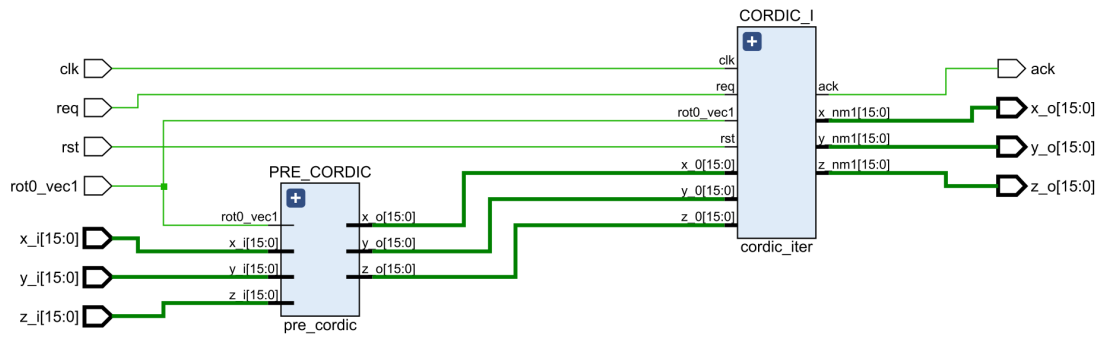


Figura 10: Figura 10

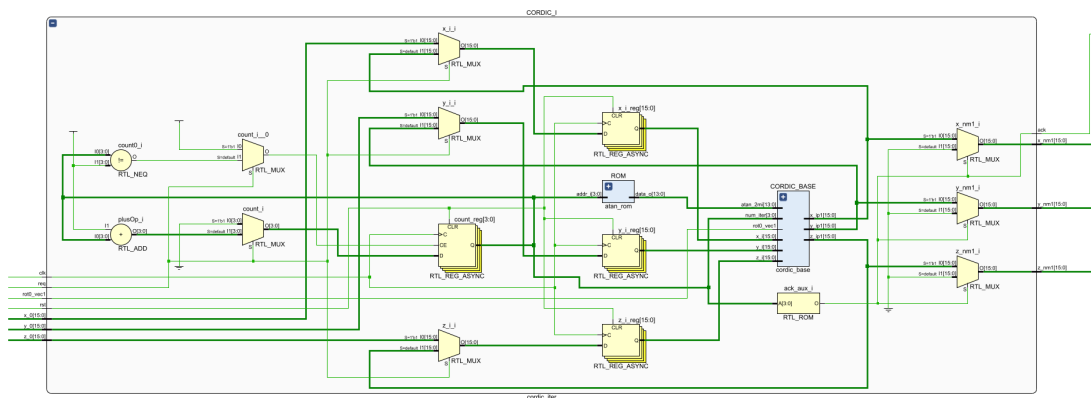


Figura 11: Figura 11

Resource	Utilization	Available	Utilization %
LUT	251	10400	2.41
FF	52	20800	0.25
IO	101	170	59.41

Figura 12: Figura 12

Setup	Hold	Pulse Width
Worst Negative Slack (WNS): inf	Worst Hold Slack (WHS): inf	Worst Pulse Width Slack (WPWS): NA
Total Negative Slack (TNS): 0,000 ns	Total Hold Slack (THS): 0,000 ns	Total Pulse Width Negative Slack (TPWS): NA
Number of Failing Endpoints: 0	Number of Failing Endpoints: 0	Number of Failing Endpoints: NA
Total Number of Endpoints: 157	Total Number of Endpoints: 157	Total Number of Endpoints: NA

Figura 13: Figura 13

5.2. Arquitectura desenrollada

El esquemático RTL se presenta en las Figuras 14 y 15, siendo esta última un acercamiento que permite observar con mayor claridad la entidad *CORDIC_UP*. Dado que dicha entidad es demasiado extensa para visualizarse en una sola imagen, se realizó un zoom sobre su primera sección con el fin de apreciar el conexionado y el esquema de *pipelining* implementado.

En cuanto al circuito de implementación completo, su tamaño imposibilita su inclusión en el cuerpo principal del informe. Por esta razón, se lo adjunta como un anexo al presente trabajo.

Finalmente, el resumen de los recursos utilizados y el de tiempos de ejecución se muestran en las Figuras 16 y 17, respectivamente.

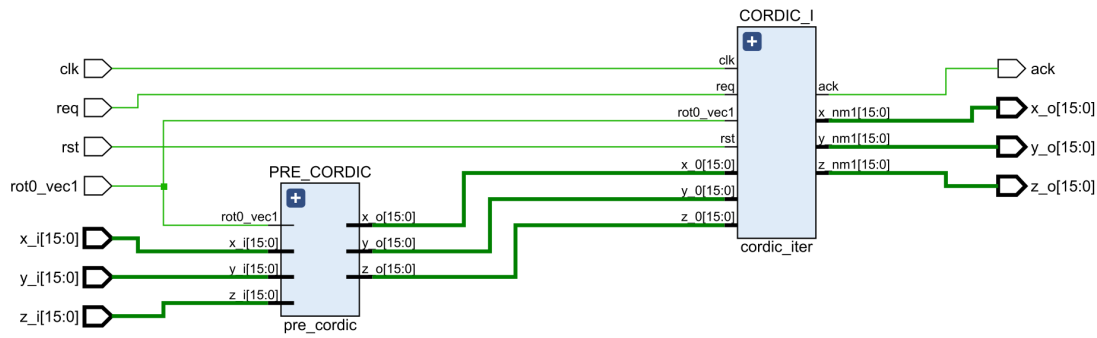


Figura 14: Figura 14

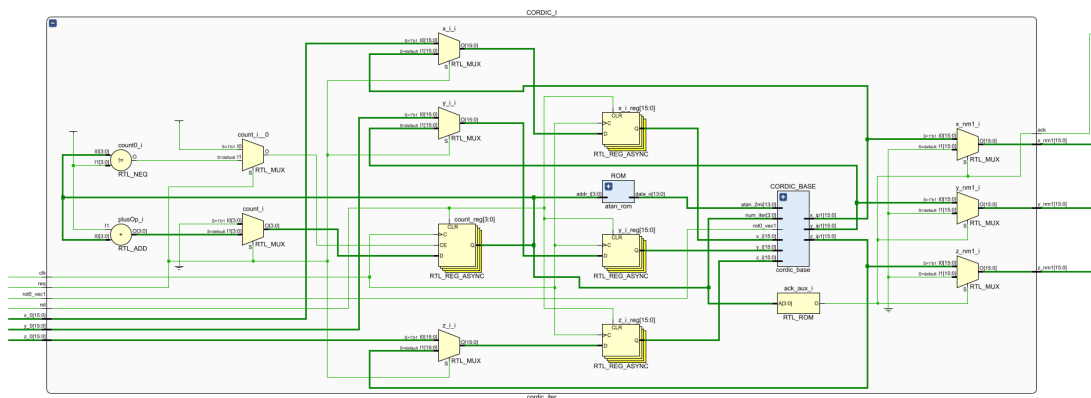


Figura 15: Figura 15

Resource	Utilization	Available	Utilization %
LUT	251	10400	2.41
FF	52	20800	0.25
IO	101	170	59.41

Figura 16: Figura 16

Setup	Hold	Pulse Width
Worst Negative Slack (WNS): inf	Worst Hold Slack (WHS): inf	Worst Pulse Width Slack (WPWS): NA
Total Negative Slack (TNS): 0,000 ns	Total Hold Slack (THS): 0,000 ns	Total Pulse Width Negative Slack (TPWS): NA
Number of Failing Endpoints: 0	Number of Failing Endpoints: 0	Number of Failing Endpoints: NA
Total Number of Endpoints: 157	Total Number of Endpoints: 157	Total Number of Endpoints: NA

Figura 17: Figura 17

6. Conclusiones

En conclusión, se logró comprender en profundidad el funcionamiento del algoritmo *CORDIC*, y se evidenció su principal ventaja: la posibilidad de implementarlo en hardware sin necesidad de multiplicadores, utilizando únicamente operaciones de suma, resta y desplazamiento.

Además, los resultados obtenidos a partir de las simulaciones muestran un error bajo, lo cual valida la correcta implementación del algoritmo. Este error puede reducirse aún más incrementando la cantidad de iteraciones o empleando unidades de punto flotante para las operaciones aritméticas.

También se analizaron las diferencias entre las dos arquitecturas desarrolladas. La arquitectura iterativa demostró una menor utilización de recursos, dado que reutiliza el módulo *CORDIC* base mediante realimentación. Sin embargo, presenta un mayor tiempo de procesamiento en comparación con la arquitectura desenrollada con *pipeline*.

Por otro lado, la arquitectura desenrollada permite una ejecución más rápida gracias al paralelismo y al *pipelining*, aunque con un mayor consumo de recursos lógicos. Esta diferencia puede observarse claramente en los informes de síntesis generados por el software *Vivado*.

7. Repositorio del Proyecto

El código fuente y archivos relacionados están disponibles en el siguiente repositorio de GitHub: [Repositorio Sistemas Digitales](#).